Міністерство освіти і науки України
Національний технічний університет
«Харківський політехнічний інститут»
Навчально-науковий інститут комп’ютерних наук та інформаційних технологій
Кафедра системного аналізу та інформаційно-аналітичних технологій

Звіт з лабораторної роботи № 3

з дисципліни ”Операційні системи та системне програмування”

Тема: Розробка програмного забезпечення, створення bash-сценаріїв та обробка сигналів в ОС Linux

Перевірив: доцент кафедри САІТ
В’ячеслав КОЛБАСІН

Виконав: студент групи КН-1225б
Артем КАРПЕНКО

Харків – 2026

Мета: Ознайомитися з процесом розробки C++ застосунків у середовищі Linux, основами створення bash-сценаріїв для автоматизації виконання та механізмами міжпроцесної взаємодії через сигнали операційної системи.

Індивідуальне завдання (Варіант 3):

1. Встановити на операційній системі Linux середовище розробки Visual Studio Code та компілятор і бібліотеки для розробки на C++.
2. Створити застосунок, який повертає як код виходу різницю довжини рядків (переданих як параметри).
3. Створити bash-сценарій для передачі аргументів до застосунку та обробки результатів його виконання.
4. Ознайомитися з прикладом застосунку, який обробляє сигнали ОС Linux (signal_example.cpp) та переробити його: зупинка після 5 натискань Ctrl-C, зупинка командою kill, надсилання сигналу Ctrl-C командою kill.
5. Створити звіт, в якому навести код та приклади його виконання.

1. Налаштування середовища розробки

У ході виконання першого пункту завдання було підготовлено робоче середовище для розробки мовою C++ у системі Linux Mint. Перевірку коректного встановлення компілятора `g++` було здійснено через вбудований термінал Visual Studio Code (рисунок 1).

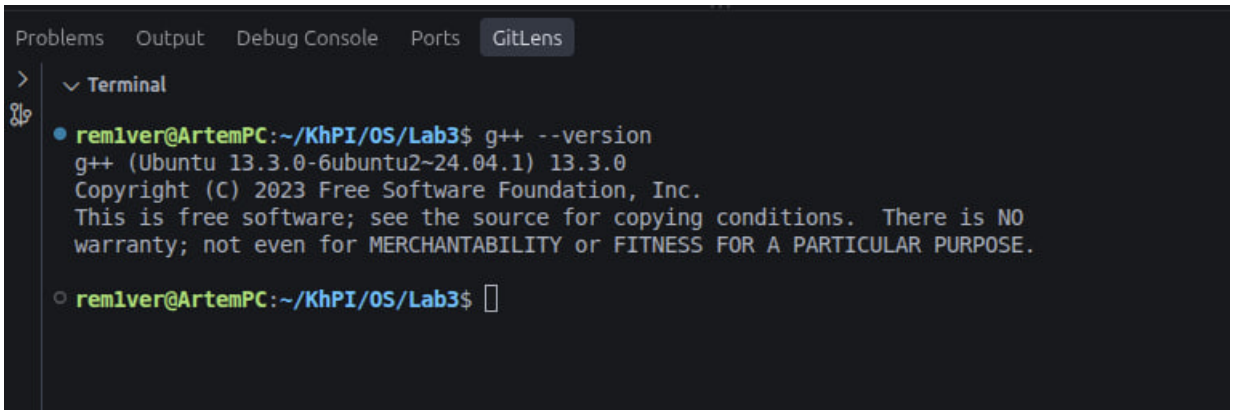


Рисунок 1 – Перевірка встановленого компілятора g++ у середовищі VS Code

2. Розробка C++ застосунку та bash-сценарію

Для вирішення індивідуального завдання (варіант 3) було створено C++ програму `app.cpp`, яка обчислює модуль різниці довжин двох рядкових

аргументів. Для реалізації цього завдання рядки зчитуються з масиву `argv`, їхні розміри визначаються методом `length()`, а різниця береться за модулем за допомогою математичної функції `std::abs`. Програма перевіряє кількість переданих параметрів через змінну `argc` і повертає відповідне числове значення як код виходу, або ж спеціальний код помилки 255 у разі невідповідності кількості аргументів (рисунок 2).

app.cpp

```cpp
#include <iostream>
#include <string>
#include <cmath>

int main(int argc, char* argv[]) {

    if (argc != 3) {
        return 0;
    }
    std::string str1 = argv[1];
    std::string str2 = argv[2];

    int diff = std::abs((int)str1.length() - (int)str2.length());
    return diff;
}
```

Рисунок 2 – Вихідний код програми app.cpp

З метою автоматизації перевірки роботи програми було розроблено сценарій командного рядка (bash). Скрипт ініціює запуск скомпільованого застосунку, перехоплює системну змінну `$?` та порівнює її з самостійно обчисленим у скрипті очікуваним результатом. Код bash-сценарію наведено на рисунку 3.

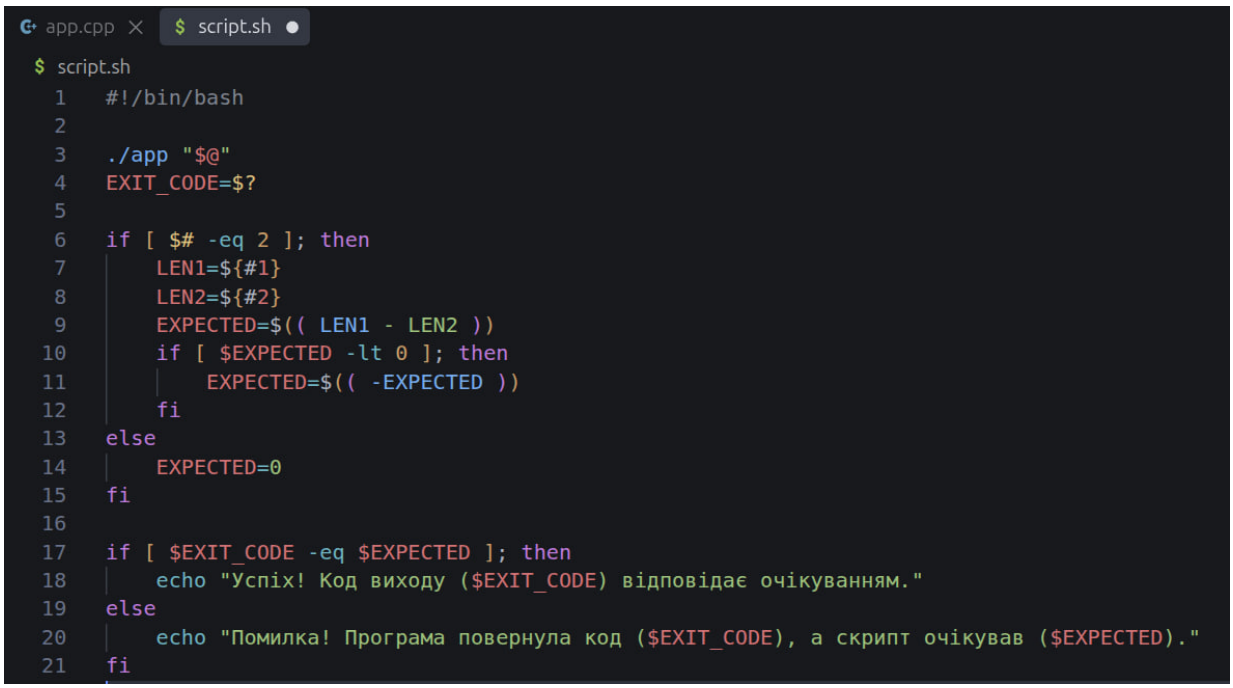


Рисунок 3 – Код bash-сценарію для автоматизованого тестування

Тестування скрипта продемонструвало його коректну роботу: при передачі двох слів різної довжини скрипт підтверджує успіх, а при помилковій кількості аргументів — фіксує конфлікт очікувань (рисунок 4).

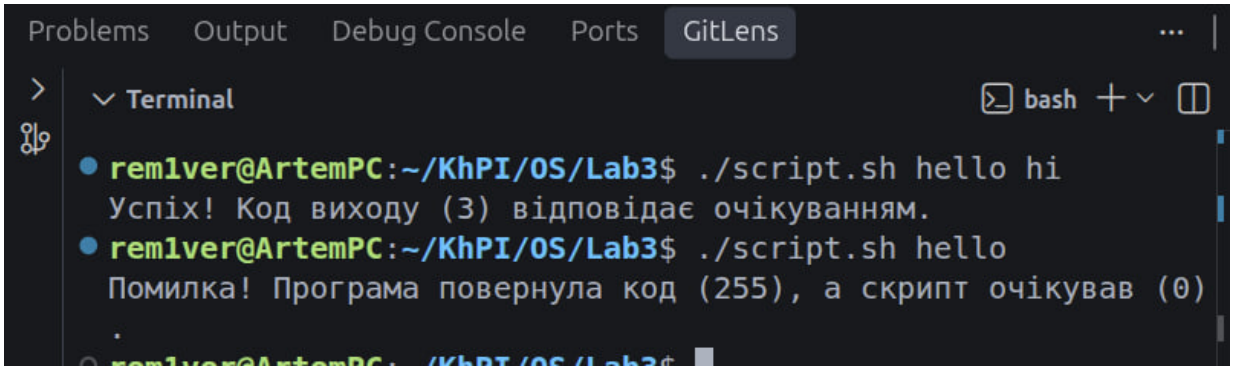


Рисунок 4 – Результати виконання bash-сценарію

3. Робота з сигналами операційної системи Linux

Наступним етапом стало дослідження міжпроцесної взаємодії через сигнали. Базовий код `signal_example.cpp` було модифіковано відповідно до вимог завдання. Зокрема, додано нову глобальну змінну типу `sig_atomic_t` для безпечного підрахунку переривань. Основні зміни торкнулися функції-обробника `handle_signal`: логіку реакції на сигнал `SIGINT` (Ctrl-C) було змінено таким чином, щоб при перехопленні лічильник збільшувався, але програма ігнорувала запит на вихід. Лише після п’ятого виклику глобальний прапорець `keep_running`

переводиться у стан 0, що змушує головний цикл `while` перерватися і дозволяє програмі штатно завершитися. Оновлений код програми наведено у лістингу 1.

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// ДОДАНО: Глобальний прапорець та лічильник натискань
volatile sig_atomic_t keep_running = 1;
volatile sig_atomic_t ctrl_c_count = 0;

// ЗМІНЕНО: Функціяобробник- сигналів
void handle_signal(int sig) {
    if (sig == SIGINT) {
        // ДОДАНО: Збільшуємо лічильник при кожному натисканні Ctrl-C
        ctrl_c_count++;
        printf("\nCaught SIGINT (Ctrl-C). Count: %d/5\n", ctrl_c_count);

        // ДОДАНО: Перевірка ліміту для зупинки
        if (ctrl_c_count >= 5) {
            printf("Limit reached. Exiting gracefully...\n");
            keep_running = 0; // Дозвіл на вихід з циклу
        }
    } else if (sig == SIGTERM) {
        printf("\nCaught SIGTERM (kill). Exiting gracefully...\n");
        keep_running = 0;
    }
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction SIGINT");
        exit(1);
    }
    if (sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("sigaction SIGTERM");
```

```
        exit(1);
    }

    printf("Process running with PID %d\n", getpid());
    printf("Press Ctrl-C or send 'kill %d' to test.\n", getpid());

    // Головний цикл очікування
    while (keep_running) {
        printf("Working...\n");
        sleep(2);
    }

    printf("Cleanup done. Bye!\n");
    return 0;
}
```

Лістинг 1: Модифікований код обробки сигналів signal_example.cpp

Роботу скомпільованого застосунку та успішну обробку п’яти сигналів переривання продемонстровано на рисунку 5.

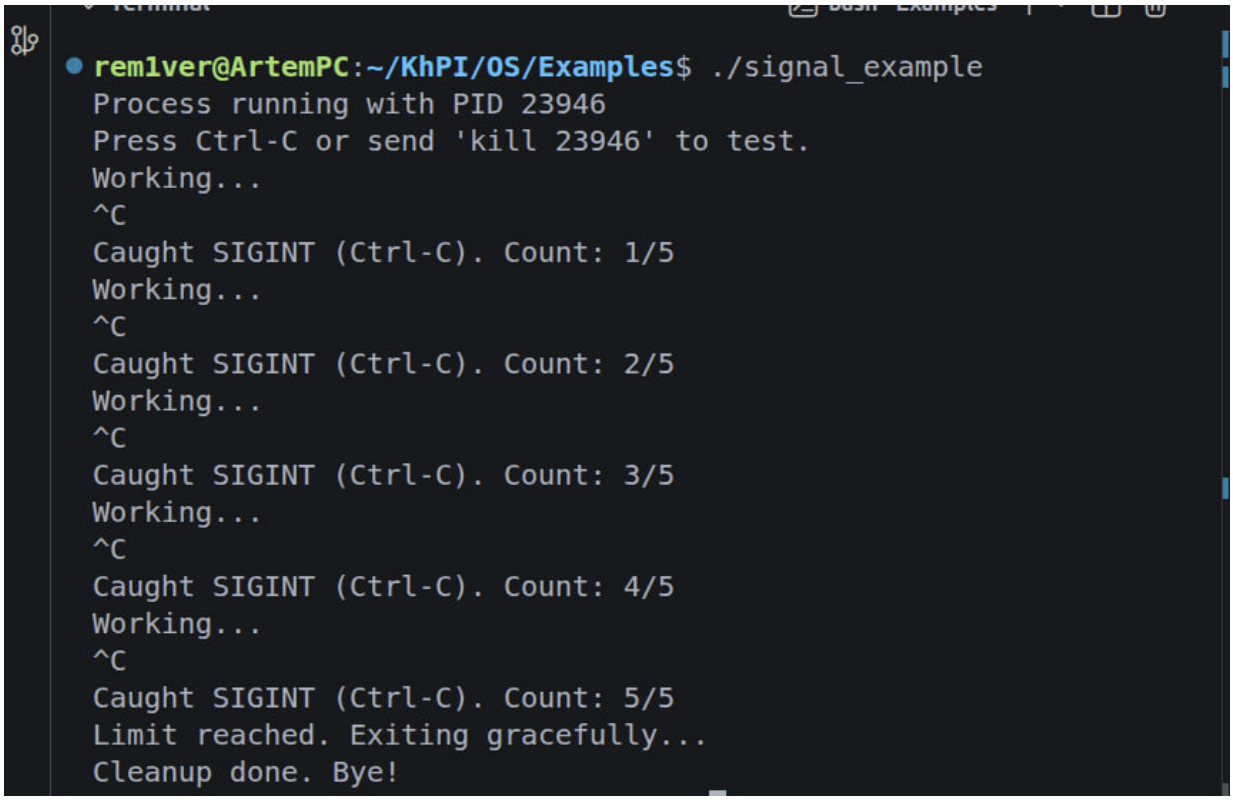


Рисунок 5 – Обробка сигналу SIGINT та штатна зупинка програми

Для перевірки можливості дистанційного управління процесами було застосовано команду `kill`. Через окреме вікно термінала було надіслано сигнал `SIGINT` за ідентифікатором процесу (PID), що продемонстровано на рисунку 6.

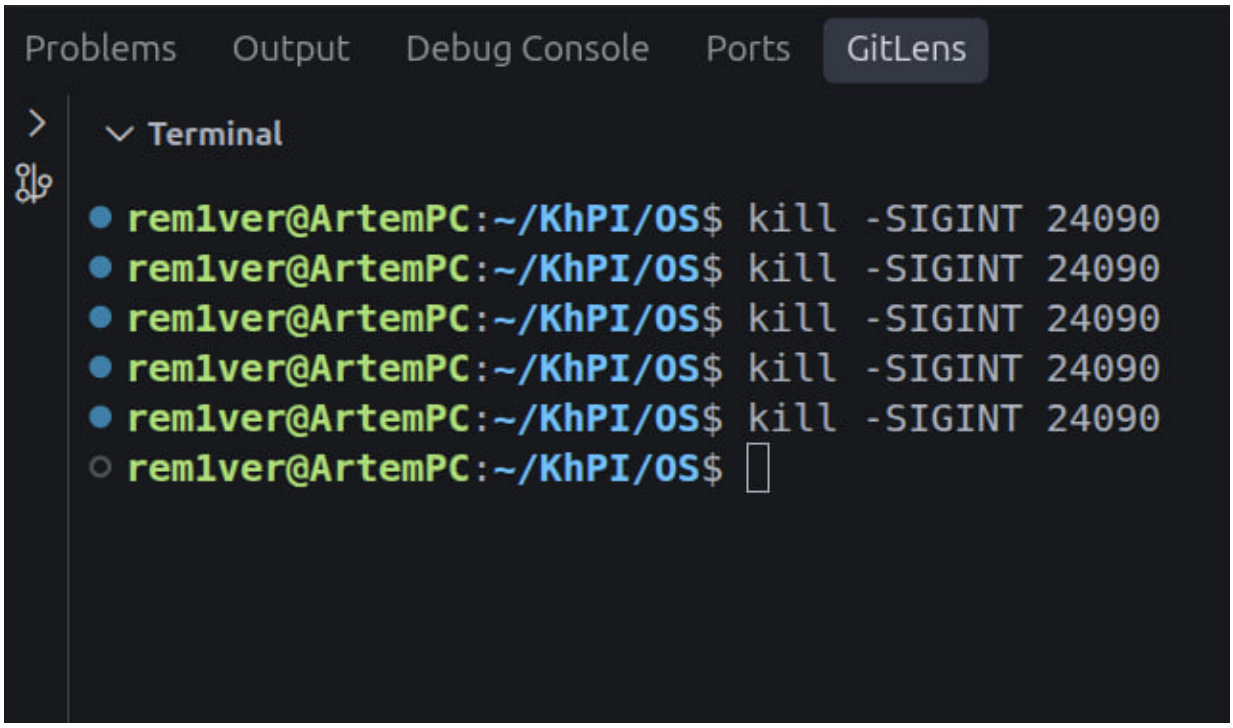


Рисунок 6 – Надсилання сигналу SIGINT за допомогою команди kill

У випадках, коли процес ігнорує стандартні запити на завершення, використовується сигнал безумовного переривання `SIGKILL` (ключ `-9`). Застосування цього підходу дозволило миттєво та примусово звільнити ресурси, про що свідчить статус ”Killed”(рисунок 7).

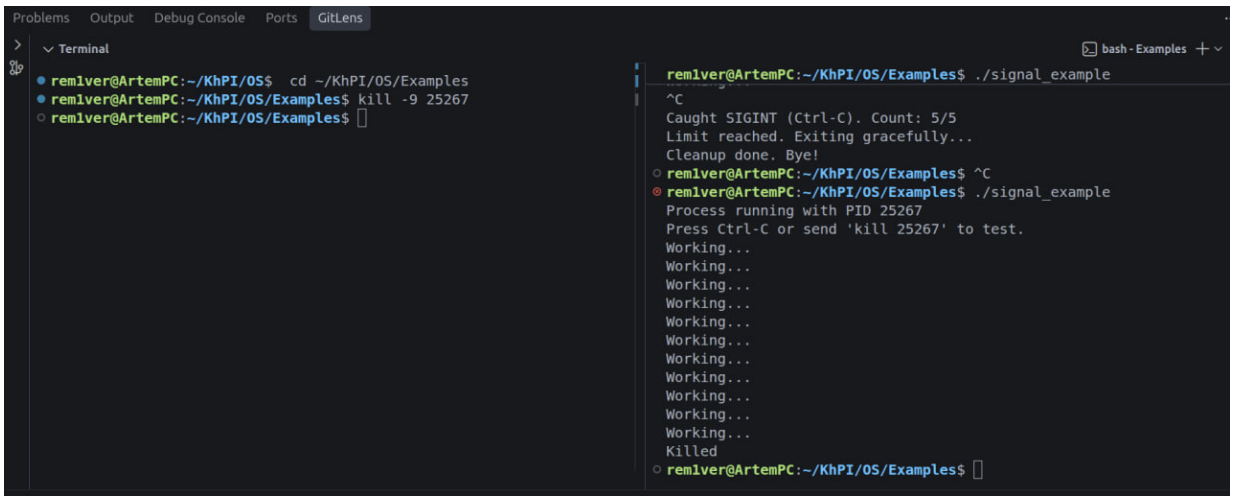


Рисунок 7 – Примусове завершення процесу командою kill -9

ВИСНОВКИ

Під час виконання лабораторної роботи було поглиблено знання щодо роботи у середовищі Linux та створення програмного забезпечення мовою C++. На практиці реалізовано:

- розробку консольного застосунку, який повертає результати обчислень (різницю довжин рядків) у вигляді системного коду виходу;
- створення bash-сценарію для автоматичного тестування програми з використанням системних змінних та умовних конструкцій;
- перехоплення та модифікацію поведінки програми при отриманні сигналів ОС (на прикладі `SIGINT`);
- управління фоновими та активними процесами за допомогою утиліти `kill` (дистанційне надсилання сигналів та примусове завершення `SIGKILL`).

Отримані навички є фундаментальними для автоматизації тестування програмного забезпечення та адміністрування процесів у UNIX-подібних операційних системах.